

To Create Intelligent Adaptive Game Opponent by Using Monte-Carlo for the Game of Pac-Man

Xiao Liu, Yao Li, Suoju He, Yiwen Fu, Jiajian Yang, Donglin Ji, Yang Chen
*International School, Beijing University of Posts and Telecommunications, Beijing, China,
 100876
 ee06b005@buptis.cn*

Abstract

Adaptive Game AI improves adaptability of opponent AI with the challenge level of the gameplay; as a result the entertainment of game is augmented. Opponent game AI is usually implemented by scripted rules in video games. However MCT (Monte-Carlo for Trees), a most updated algorithm which perform excellent in computer go can also be used to achieve excellent result to control non-player characters (NPCs) in video games. In this paper, the prey and predator game genre of Pac-Man is used as a test-bed, the basic principle of MCT is presented, and the effectiveness of its application to game AI development is demonstrated. Furthermore, in order to reduce the computation intensiveness of Monte-Carlo, ANN (Artificial Neural Network) is used to produce the intelligence of game opponent with the data collected from Monte-Carlo method. The effectiveness and efficiency of the process is proved.

Keywords: Adaptive Game AI, Monte-Carlo, MCT, Pac-Man

1. Introduction

Adaptive Game AI could improve adaptability of opponent AI through adjusting the challenge level by improve or lower intelligence of NPCs automatically; as a result, the entertainment of game is promoted. However, opponent game AI nowadays is usually implemented by scripted rules in video games. And because scripted rules are hard coded, the intelligence of opponent is difficult to be adjusted. Therefore, the most updated algorithm of Monte-Carlo which based on computation is used in this paper to create a controller for NPCs. Monte-Carlo implemented opponent can not only produce high-level opponent AI but also achieve good adaptability which contributes

well to the entertainment of game as it dynamically adjusts to different gamers' needs.

The paper continues in the next section with the description of game Pac-Man; while Section 3 explains Monte-Carlo algorithms; Section 4 demonstrates how to apply Monte-Carlo algorithm to improve the adaptability of the game Pac-Man; and finally in Section 5 conclusions are drawn.

2. Description of game Pac-Man

Pac-Man (パッくマン, Pakkuman) is an arcade game developed by Namco and licensed for distribution in the U.S. by Midway, first released in Japan on May 22, 1980. Pac-Man is universally considered as one of the classics of the medium, virtually synonymous with video games, and an icon of 1980s popular culture [1].



Figure 1. Game of Pac-Man

As Pac-Man is a linear and relatively simple game which can present the principle of video game, it is used as a test-bed for the implementation of the experiments. However, to reduce the complexity of the game and facilitate the experiment, the rules of the

game are modified. Thus we provide a description of the game mechanics below [2].

- The game world is a 2D graph, where the nodes of the graph can be Dot or empty (Power up is removed from original game reduce complexity).
- Pac-Man and the Ghosts (the in-game actors) move between nodes along two axes, horizontal and vertical.
- Ghosts, starting in a central box area, aim at chasing and finally eating Pac-Man.
- Pac-Man, starting at a stationary chosen position, aims at eating Dots or survives for a predetermined large period of time.
- Pac-Man Eats Dots when he Moves over them.
- The intention of Ghosts is to eat Pac-Man. After Pac-Man is eaten, Ghosts Win, and the game is over.
- Pac-Man must eat 80 Dots to Win.
- If 100 steps have been finished and still neither Pac-Man nor Ghosts Win, then the outcome of the game is considered as a Draw.
- Ghosts are permeable and do not interact with Dots or obstruct each other.
- In order to improve the game balance, two Ghosts are designed and their speeds are as same as Pac-Man. So it is impossible for a single Ghost to finish the task of eating Pac-Man (cooperated agent is required).

3. Background of Monte-Carlo

Monte-Carlo is a learning method for estimating value functions and discovering optimal policies. Monte-Carlo methods require only experience-sample sequences of actions, and rewards from simulated interaction with an environment. Learning from simulation is striking because it requires no prior knowledge of the environment's dynamics, yet can still attain optimal behavior [3].

Monte-Carlo method is a computational algorithm that relies on repeated random sampling to compute their results. It tends to be used when it is infeasible or impossible to compute an exact result with a deterministic algorithm [6]. Because of their reliance on repeated computation and random or pseudo-random numbers, the performance of Monte-Carlo method is proportional to the computing capability of a computer.

4. Apply Monte-Carlo algorithm to improve the adaptability of the opponent AI of the game Pac-Man

This section demonstrates how to apply Monte-Carlo algorithm to improve the adaptability of the

game Pac-Man. Subsection 4.1 presents the application of Monte-Carlo to Pac-Man; Subsection 4.2 illustrates the adaptability of opponent AI from the relationship between the performance of Monte-Carlo and its computation time limit; while the illustration how Neuro-controller is built with the data collected from Monte-Carlo method to reduce the intensiveness of Monte-Carlo is presented in Subsection 4.3.

4.1. Application of Monte-Carlo to Pac-Man

The application of Monte-Carlo for Pac-Man is based on the hypothesis that to control each Ghost in game board situation is actually determined by a choice of legal move [5]. The possible legal moves could be in the direction of North, South, East, and West.

The possible moves of Ghost at each simulation are determined as follow: first according to the domain knowledge of Pac-Man, choose Ghost's possible directions where there is no walls from all directions; then use domain knowledge and experience to filter out illegal and inefficient moves of Ghost according to the former experience, as a result the possible legal moves is reduced to less than four; finally, choose all legal moves as first step in simulation one after another, and the moves for the rest of steps are chosen randomly. The result of the simulation will be returned as supervision to create intelligence for Ghosts.

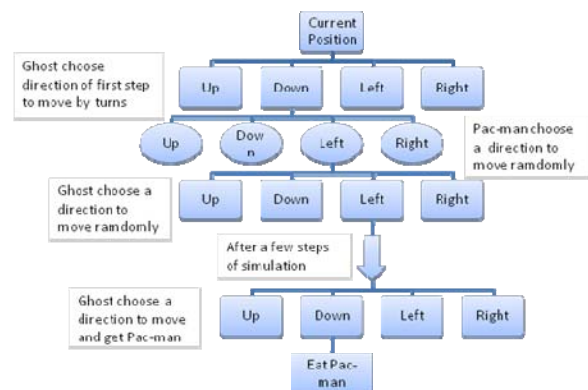


Figure 2. Simulation Process (Tree search)

In simulation process of every step, the Monte-Carlo algorithm builds a search tree consisted of possible legal moves. However, searching every branch in the tree is inefficient due to the intensiveness of computation. So it is indispensable to prune the tree by removing branches based on the previous experience. As a result, the search space is reduced, and the efficiency is enhanced.

4.2. Improve adaptability of opponent AI from the relation between the performance of Monte-Carlo and its computation time limit

As Monte-Carlo is an algorithm based on computation, its performance relies on time of simulation, we consider using this property to improve or lower the performance of Monte-Carlo method for the use of adjusting the difficulty of the game.

The experiments setup and the results present as following. The simulated human player plays the game in three strategies which are Strategy A, Strategy B, and Strategy C. The NPCs or Ghosts are controlled by Monte-Carlo method. By adjusting the simulation time of Ghosts, the relation between the simulation time and win rate which represents the performance of opponent AI could be obtained. Then by applying function fitting, the function of the curve can be derived. The desired computation time limit is under 400 milliseconds, since the performance become stable with the increment of simulation time and the simulation time is too long to be tolerated by player.

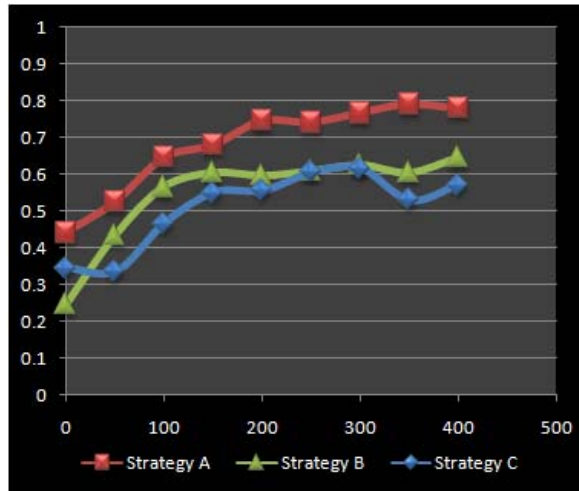


Figure 3. Performance of Monte-Carlo with its computation time limit

From the curve in Figure.3, using Strategy A as an example, we can see that in the time interval from 0ms to 200ms, the win rate which represents the performance of Monte-Carlo method increases with respect to the increment of simulation time. However, from 200ms, the performance become stable and cannot be improved which we believe is for the nature of Monte-Carlo method.

Monte-Carlo is one of random methods of computation and experimentation generally considered forms of stochastic simulation [6]. At the beginning interval with increment of simulation time, the sample

space become more abundant, the stochastic results become more and more precise, so the performance of Monte-Carlo method improved sharply. But after a quantity of increment of simulation time, the stochastic results are precise enough for the Ghosts to make the decision (which direction to choose), so the more precise solutions are dispensable.

Another reason that the performance cannot be improved more is that Monte-Carlo method has an innate defect. "This approach sometimes misses an isolated, very good move. These moves are often strong strategically but weak tactically, as tactical decisions tend to rely on a small number of crucial moves which are easily missed by the randomly searching Monte-Carlo algorithm" [6]. For example, Ghost now has a move in one choice of directions which may absolutely lead to win, but in all the possible moves from this direction, this move is so rare that it will not influence the win rate of this direction. As a result, other direction may be chosen and this perfect move will never be discovered.

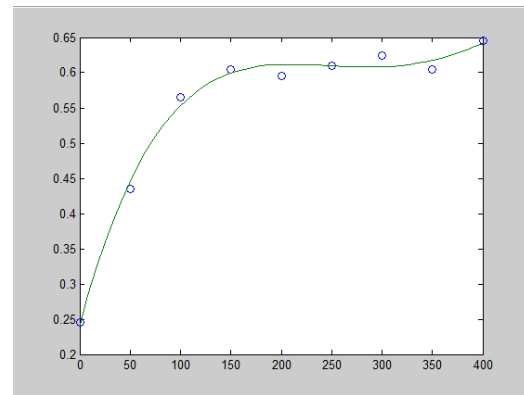


Figure 4. Function fitting process

Strategy 2 is used as an example for the elaboration of how function fitting is implemented. And we use Matlab a powerful mathematical tool for the implementation. Figure.4 shows the result of function fitting and code provided as following:

```
x=[0 50 100 150 200 250 300 350 400];
y=[0.245 0.435 0.565 0.605 0.595 0.61 0.625 0.605
0.645];
p=polyfit(x,y,5)
xi=linspace(0,400,50);
yi=polyval(p,xi);
plot(x,y,'o',xi,yi)
```

The fitting function is automatically generated by Matlab:

$$y=0.0000x^5-0.0000x^4+0.0000x^3-0.0000x^2+0.0053x+0.2424$$

From the function, the performance of opponent AI can be adjusted gradually and precisely.

After all, the adaptability of opponent AI is presented, and the difficulty of the game is adjustable corresponding to the opponent AI. Once the simulation time is configured, the difficulty of the game is confirmed.

4.3. Training Neuro-controller from Monte-Carlo created sample data

Though Monte-Carlo method can create near optimal control for Pac-Man or analogous games, it is actually not suitable for similar online Games or running in some unsatisfactory environment due to computational intensiveness of Monte-Carlo. However, as Monte-Carlo can create near optimal control data, it is possible to create Neuro-controlled game opponent by learning off-line from the Monte-Carlo created sample data [4]. Finally Neuro-controlled game opponent for online Games from Monte-Carlo created data without worry about computational intensiveness is generated. In the experiment we use the widely used model of 3-Layered Feed-Forward Artificial Neural Network in Weka for implementation.

The sample data are from 89 of gameplays with a total of 4723 instances. However the data are optimized by only keeping the instances belonged to the winning opponent. The saved instances are used to train the Neural Controller [4].

In this experiment, Strategy B of simulated human player is chosen for instance. The 22 variables chosen from the Ghosts and Pac-Man's relationship with game environment as the inputs of the ANNs are:

stepNum,
playerX,
playerY,
playerDirection,
playerDotUp,
playerDotDown,
playerDotLeft,
playerDotRight,
ghost0X,
ghost0Y,
ghost0WallUp,
ghost0WallDown,
ghost0WallLeft,
ghost0WallRight,
ghost1X,
ghost1Y,
ghostsDistanceX,
ghostsDistanceY,
ghost0PlyaeDistanceX,
ghost0PlyaeDistanceY,

ghost1PlyaeDistanceX,
ghost1PlyaeDistanceY,

Four inputs (ghost0WallUp, ghost0WallDown, ghost0WallLeft, ghost0WallRight) do not connect to hidden layer, because these four factors are not "considerable" conditions but limited condition, so they are connected to output layer directly. Furthermore, the inputs of the second Ghost are slightly different with the first one. It has four more inputs (ghost0Up, ghost0Down, ghost0Left, ghost0Right) which indicate the choice of first Ghost. These four inputs ensure that the two Ghosts are cooperating.

The variables chosen as the outputs of the ANNs which determine Ghost's direction of movement are: North, South, East, and West.

Choice of number of hidden layer is 11 which is equal to the sum of 1/2 of the total of number of input nodes (exclude 4 nodes which are not connected to hidden layer) and number of output node when the ANNs are found to perform the best.

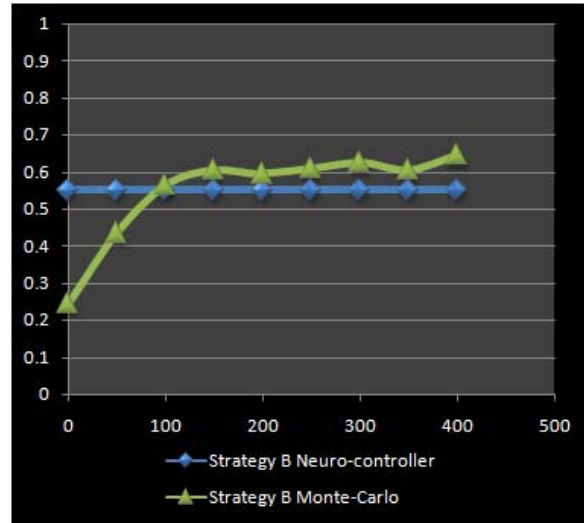


Figure 5. Performance comparison: Monte-Carlo and Neuro-controller

From patterns in Figure.5, we can see, compare to Monte-Carlo method, the performance of Neuro-controller is fixed even the computing environment is unsatisfied.

5. Conclusion

From the result in Figure.3 in Subsection 4.2, we can conclude that with the time for each simulation augment to 400 milliseconds, the performance of opponent AI shown for Strategy A, B, and C all

improve with the length of the simulation time increases. Unfortunately if the simulation time is more than 400 milliseconds, it is usually human gamer intolerable. What's more, as the Monte-Carlo is much more computational expensive which prohibit it from using for online games; while the trained Neural Controller cost almost no time for computation, so it is also a good choice of implementation for opponent AI.

The approach proposed in this paper may be extendable to other Real-Time game genres from the inspiration of this paper.

6. Acknowledgement

This research is supervised and reviewed by Suoju He; it is based on the previous research of Suoju He and Fan Xie.

This paper is funded by International School, Beijing University of Posts and Telecommunications. I am very grateful for the financial support.

References

- [1] Pac-Man, Wikipedia, Retrieved from <http://en.wikipedia.org/wiki/Pac-Man> (03/08/09).
- [2] Ben Cowley, Darryl Charles, Michaela Black, Ray Hickey, "Data-Driven Decision Theory for Player Analysis in Pacman" Association for the Advancement of Artificial Intelligence, 2007.
- [3] Richard S. Sutton, Andrew G. Barto, Reinforcement Learning: An Introduction.
- [4] Fan Xie, Suoju He, Xiao Liu, Xingguo Li, Junping Du, Jiajian Yang, Yiwen Fu, Yang Chen, Junping Wang, Zhiqing Liu, Qiliang Zhu, "To Create Neuro-Controlled Game Opponent from UCT Created Data", GEC'09, June 12-14, 2009.
- [5] Suoju He, Fan Xie, Yi Wang, Sai Luo, Yiwen Fu, Jiajian Yang, Zhiqing Liu, Qiliang Zhu, "To Create Adaptive Game Opponent by Using UCT".
- [6] Monte-Carlo method, Wikipedia, Retrieved from http://en.wikipedia.org/wiki/Monte_Carlo_method (03/08/09).
- [7] Yannakakis, Giorgios N, "AI in Computer Games: Generating Interesting Interactive Opponents by the use of Evolutionary Computation". PhD thesis, University of Edinburgh, 2005.
- [8] Georgios N. Yannakakis, John Levine, and John Hallam. "An Evolutionary Approach for Interactive Computer Games", In Proceedings of the Congress on Evolutionary Computation (CEC-04), pages 986–993, June 2004.
- [9] Peter Bentley Article, "Letting Stones Go Unturned", Feb, 2008.
- [10] Simon M. Lucas, "Computational Intelligence and Games: Challenges and Opportunities", International Journal of Automation and Computing, Pages 45-57, January, 2008.
- [11] Levente Kocsis and Csaba Szepesvári, "Bandit based monte-carlo planning". In 15th European Conference on Machine Learning (ECML), pages 282–293, 2006.
- [12] Suoju He, Fan Xie, Yi Wang, Jin Meng, Hongtao Chen, Zhiqing Liu, Qiliang Zhu, "Game Player Strategy Pattern Recognition and How UCT Algorithms Apply Pre-Knowledge of Player's Strategy to Improve Opponent AI", In the International Conference on Innovation in Software Engineering (ISE'2008), 2008.